Programme de colle - Semaine 9 (du 2 novembre)

La colle est constituée d'une démonstration ou exercice de cours (parmi les énoncés ci-dessous marqués par une (*)) (20min max) suivi d'un ou plusieurs exercices portant sur le programme ci-dessous.

1 Langages formels. Langages réguliers

- Mots : alphabet, mot sur un alphabet, concaténation, puissance d'un mot, structure de monoïde $(\Sigma^*, .)$.
- Préfixes, suffixes, facteurs, sous-mots.
- Langages : opérations ensemblistes sur les langages (union, intersection, différence, complémentaire), concaténation de deux langages, étoile de Kleene d'un langage,
- Langages réguliers (aussi appelés langages rationnels): définition inductive de la classe des langages réguliers, **tout langage fini est régulier (*)**
- Expressions régulières : définition inductive, langage dénoté par une expression régulière (notation $\mu(e)$)
- **Un langage est régulier si et seulement si il existe une expression régulière qui le dénote (*)** [NB : interroger sur un seul sens uniquement, chaque sens se démontre par induction]
- Expressions régulières équivalentes
- Expressions régulières étendues : par exemple $e?$, $e+$, $e\{3, 5\}$, etc. Aucune syntaxe n'est exigible mais il faut savoir comprendre une expression régulière étendue et la traduire en expression régulière du cours équivalente (ex : $e? \equiv (e|\varepsilon)$).

2 Automates finis

- **Automates finis déterministes (afd)**

- Définis comme un quadruplet (Q, q_0, F, δ) avec Q ensemble des états, q_0 état initial, F états finaux, $\delta : Q \times \Sigma \to Q$ fonction (partielle) de transition.
- Définition, notion de calcul dans un afd, notion de blocage.
- Langage reconnu (ou accepté) par automate : calcul réussi, mots acceptés. Classe des langages reconnaissables notée $\mathrm{REC}(\Sigma^*)$.
- Accessibilité et co-accessibilité; afd émondé
- Afd complet; complétion d'un afd
- **Automate complémentaire : Si L est reconnaissable alors $\bar{L}$ l'est (*)**
- Automate produit. **Si A_1 et A_2 sont 2 afd alors $\mathcal{L}(A_1 \times A_2) = \mathcal{L}(A_1) \cap \mathcal{L}(A_2)$ (*)**.

- **Automates finis non déterministes (afnd)**

  - Définis comme un quadruplet (Q, I, F, T) avec Q ensemble des états, I états initiaux, F états finaux, $T \subset Q \times \Sigma \times Q$ ensemble des transitions.
  - Calcul dans un afnd. Calcul réussi.
  - Mots acceptés par un afnd. Langage reconnu.
  - Fonction de transition $\delta : Q \times \Sigma \to \mathfrak{P}(Q)$ et fonction de transition étendue $\delta : Q \times \Sigma^* \to \mathfrak{P}(Q)$ d'un afnd :

    $$\delta(q, c) = \{q' \ / \ (q, c, q') \in T\}.$$

    La notion de blocage se traduit par $\delta(q, c) = \varnothing$.
  - Automate des parties. Algorithme de déterminisation d'un afnd.
  - Les notions suivantes restent valides : états accessibles, co-accessibles, automate émondé, complétion. **L'algorithme de l'automate complétaire ne fonctionne pas sur un automate non déterministe** (déterminiser d'abord).
  - Définition de l'automate produit dans le cas d'un automate non déterministe. L'algorithme de construction pas à pas ne change pas : on fait simplement attention à ce que la lecture d'une lettre depuis un état peut conduire à plusieurs transitions.

- **Automates finis non déterministes à transitions spontanées (ε-afnd)**

  - Définis comme un quadruplet (Q, I, F, T) avec Q ensemble des états, I états initiaux, F états finaux, $T \subset Q \times \Sigma \cup \{\varepsilon\} \times Q$ ensemble des transitions.
  - Les notions de calcul, calcul réussi, mot accepté, langage reconnu, *etc* sont les mêmes que pour un afnd.
  - ε-clôture ou ε-fermeture, notée $\eta : Q \to \mathfrak{P}(Q)$ dans le cours, définie comme l'ensemble des états qu'on peut atteindre à partir d'un état q en n'empruntant que des ε-transitions.
  - Formules récursives pour le calcul de la fonction de transition étendue : $\delta : Q \times \Sigma^* \to \mathfrak{P}(Q)$
  - Algorithme de suppression des ε-transitions pour passer d'un ε-afnd à un afnd équivalent (sans avoir à déterminiser).

- Algorithme de déterminisation d'un ε-afnd.

- **Langages non reconnaissables** : le lemme de l'étoile et sa démonstration doivent être sus, la méthode pour montrer qu'un langage est non reconnaissable doit être sue.

  - **Lemme de l'étoile (*)** : Soit L est un langage reconnu par un automate à n états et $u \in L$ tel que $|u| \geq n$ alors il existe une décomposition $u = xyz$ telle que $|xz| \leq n$, $y \neq \varepsilon$, $\forall k \in \mathbb{N}, xy^k z \in L$.

  - Application : **$L = \{a^n b^n, n \in \mathbb{N}\}$ n'est pas reconnaissable (*)**

  - Utilisation des propriétés de fermeture des langages reconnaissables pour montrer qu'un langage n'est pas reconnaissable.

NB : le lien entre langages réguliers et langages reconnaissables n'a pas encore été étudié : pas d'automates de Thomson à ce stade

3 Arbres couvrants de poids minimal

- **Structure de données Union-Find (Unir et Trouver)** : sert à représenter une partition de $[\![0, N-1]\!]$.

  - Implémentation favorisant `Find` : implémentation naïve sous forme d'un tableau `T` dans lequel `T[i]` est le représentant de l'élément i. Le `Find` est en $O(1)$, le `Union` est en $O(N)$.

  - Implémentation favorisant `Union` : implémentation sous forme de forêt, les éléments sont des nœuds, les parties sont les arbres de la forêt (les composantes connexes), les représentants des parties sont les racines des arbres.

    - ⋆ Optimisation 1 (utilisation des hauteurs) : lors de l'union de deux arbres on place l'arbre de plus petite hauteur sous la racine de l'arbre de plus grande hauteur.

    - ⋆ **Avec Optimisation 1, les arbres ont tous une hauteur h vérifiant $h \leq \log_2(N)$. (*)** (Preuve par invariant)

    - ⋆ Optimisation 2 (compression des chemins) : lors d'une opération find on change le parent de chacun des noeuds rencontrés pour les faire pointer directement sur la racine.

    - ⋆ Avec Optimisations 1+2 : on admet que la complexité de `Union` et de `Find` est en $O(1)$ en pratique.

- **Arbres couvrants** dans un graphe connexe : rappels des définitions de cycle et d'arbres. Définition d'arbre couvrant. Construction d'un arbre couvrant en ajoutant une à une des arêtes sans créer de cycle. Utilisation de Union-Find dans ce cadre.

- **Algorithme de Kruskal** : savoir appliquer l'algorithme et le décrire en pseudo-code avec la structure de donnée Union-Find. Complexité dominée par le tri des arêtes.